8.4 Adding ASP.NET Core framework services to the container

Before ASP.NET Core, using DI was optional. By contrast, to build all but the most trivial ASP.NET Core apps, some degree of DI is required. As I've mentioned, the underlying framework depends on it, so features such as Razor Pages and authentication require you to configure the required services. In this section you'll see how to register these framework services with the built-in container. In chapter 9 you'll learn how to register your *own* services with the DI container.

ASP.NET Core uses DI to configure both its internal components, such as the Kestrel web server, and extra features, such as Razor Pages. To use these components at runtime, the DI container needs to know about all the classes it will need. You register these services with the `Services` property on the `WebApplicationBuilder` instance in Program.cs.

NOTE The `Services` property of `WebApplicationBuilder` is of type `IServiceCollection`. This is where you register the collection of services that the DI container knows about.

If you're thinking "Wait—I have to configure all the internal components myself?", don't panic. Most of the core services are registered automatically by `WebApplicationBuilder`, and you don't need to do anything else. To use other features, such as Razor Pages or authentication, you *do* need to register the components explicitly with the container in your app, but that's not as hard as it sounds. All the common libraries you use expose handy extension methods to take care of the nitty-gritty details. These extension methods configure everything you need in one fell swoop instead of leaving you to wire everything up manually.